

Near-duplicate leakage can reorder model rankings on a genomic benchmark suite: an audit of Genomic Benchmarks

Rikhin Kavuru^{1,*}

¹Independent Researcher

*Corresponding author. rikthinkavuru@gmail.com

Abstract

Motivation: Benchmarks for DNA sequence classification underpin method comparison, but their train/test splits rarely control for near-duplicate sequences spanning the split, letting a model score well by recognizing memorized near-copies rather than by generalizing. Whether this merely inflates scores, or changes which method appears best, has not been characterized on a widely used suite.

Results: We audit the seven binary datasets of Genomic Benchmarks: two are leaky, four clean under both a k -mer and a length-robust containment metric, and one borderline. A near-duplicate-aware re-split lowers the best model's accuracy by 11.8 points on human_nontata_promoters and 16.5 on human_enhancers_ensembl; cluster-bootstrap intervals exclude zero, and every clean-dataset delta includes zero. On human_enhancers_ensembl the leading model changes under accuracy, AUROC and F_1 alike, and under a chromosome-holdout control. Across a nine-learner roster the leaky split cannot separate its top three—a perceptron, 1-nearest-neighbour and extremely randomized trees finish within 0.0015—yet those three span 23 points once corrected; 1-nearest-neighbour falls from rank 2 to last, 0.873 to 0.537. On human_nontata_promoters the demotion is accuracy-only, so the reordering is demonstrated on one dataset and cautionary on the other. The cause is a curation defect we can manipulate: the leaky positive class ships 77,421 intervals on only 40,934 distinct coordinates, and editing coordinates alone abolishes the reordering and manufactures it in a clean dataset. Three of twelve Nucleotide Transformer tasks are similarly leaky, one at 25% byte-identical, though no ranking there inverts.

Availability and implementation: Code, the report card, and a dependency-free near-duplicate-aware splitter and certification tool are at <https://github.com/rikthinkavuru/homology-leakage-genomic-benchmarks>. The classical audit is CPU-only and deterministic.

Keywords data leakage, near-duplicate sequences, benchmark evaluation, genomics, model ranking

1. Introduction

Machine learning for DNA sequence classification is now routine, and benchmark suites have become the default proving grounds for new methods. The Genomic Benchmarks collection (Grešová et al., 2023) is widely used precisely because it is small, accessible, and quick to run, making it a natural first stop when comparing models. The scientific value of any such suite, however, rests entirely on one assumption: that its train/test split measures generalization rather than recall of the training data.

Near-duplicate leakage threatens this assumption. We use *near-duplicate leakage* to mean the situation in which exact or near-identical sequences appear on both sides of a split; where local-alignment or containment evidence earns the stronger word, we call the shared-ancestry variant *homology*. A model can then succeed on the test set by recognizing memorized near-copies of training examples rather than by learning generalizable signal, so reported accuracy overstates true generalization. This failure mode is increasingly documented across biological

machine learning, from regulatory-genomics regression to protein-interaction prediction and virtual screening (Section 2).

Two claims must be kept separate. **Claim I**—that de-leaking a split lowers apparent accuracy, i.e. that leakage inflates scores—is not our contribution: it is already established across domains by prior work, including the general leakage-and-reproducibility analysis of Kapoor and Narayanan (2023), the long-range genomic regression study of Rafi et al. (2025), and the protein-interaction benchmarks of Bushuiev et al. (2024). We reproduce it here only as a within-suite control.

Claim II—that near-duplicate leakage can *reorder* which model appears best—is our contribution, and it is scoped: it requires both short, fixed- or near-fixed-length human regulatory DNA and a memorization-prone model in the comparison set. We demonstrate Claim II cleanly on one dataset and, honestly, only partially on a second.

Our contributions are:

- a per-dataset near-duplicate *report card* for the binary suite with a matched clean-versus-leaky control, showing

that leakage is dataset-specific rather than universal, and that a length-robust containment re-measure both confirms four clean verdicts and flags one borderline dataset the length-blind metric misses;

- evidence that near-duplicate leakage can *reorder the model ranking* when a memorization-prone model is in the comparison set—demonstrated on *human_enhancers_ensembl*, where accuracy, AUROC, F_1 , a chromosome-holdout control and a bootstrap winner probability all agree, and a cautionary partial case on *human_nontata_promoters*;
- a mechanism, validated three ways: a graded memorization analysis plus a random-forest regularization path (the effect is a property of the *default* unregularized forest), a from-scratch 1D CNN grid (the effect is not a classical-model artifact), and a chromosome-holdout and alignment-based re-measure (the effect is not a clustering or k -mer artifact);
- a nine-model roster spanning 1-nearest-neighbour to naive Bayes, showing the reordering is not an artifact of a four-model comparison: the leaky split ties its top three models within 0.0015 while the corrected split spreads them over 23 points, and the four models whose drops exclude zero are the three memorization-prone learners plus the perceptron, while both linear models, boosting and naive Bayes are null;
- a *cause*, identified in the shipped genomic coordinates and then manipulated: the leaky positive class is an unduplicated assembly shipping 77,421 intervals on 40,934 distinct coordinates, and editing that construction step alone abolishes the reordering in the leaky dataset and manufactures it in a clean one;
- a cross-suite check on the Nucleotide Transformer downstream tasks, where three of twelve independent tasks carry the same defect but no ranking inverts—a null whose reason is diagnosable in advance from the as-shipped split alone; and
- a dependency-free, near-duplicate-aware splitter and an executable certification tool, with an honest statement of scope.

The classical audit is deterministic, CPU-only, and reproducible; the deep-model checks are pre-registered and run deterministically on a single device.

2. Related work

Leakage is an established benchmarking hazard. Kapoor and Narayanan (2023) document leakage as a pervasive driver of the reproducibility crisis in ML-based science and show that it can inflate reported performance and, in extreme cases, change the apparent ordering of methods; our study is a genomics-specific, controlled instance of that phenomenon. In genomics, hashFrag (Rafi et al., 2025) shows that homologous sequences spanning genomic train/test splits inflate the apparent performance of neural models—test performance scales with similarity to the training set—and provides tools to detect and partition such data. In protein–protein interaction prediction, Bushuiev et al. (2024) show that standard splits leave most test cases with near-duplicate counterparts, rewarding overfitting capacity rather than generalization; PINDER (Kovtun et al., 2024) similarly shows that de-leaked test sets sharply reduce reported

performance, including for AlphaFold-Multimer. In virtual screening, an audit of LIT-PCBA (Huang et al., 2025) shows that a trivial memorization baseline can match sophisticated methods once leakage is accounted for.

The closest methodological prior art is homology-aware partitioning. GraphPart (Teufel et al., 2023) partitions biological sequences so that no train/test pair exceeds a similarity threshold, and CD-HIT (Li and Godzik, 2006) and MMseqs2 (Steinegger and Söding, 2017) are the established clustering engines for this task. Our near-duplicate-aware splitter is a lightweight, dependency-free instance of the same whole-cluster idea; we validate against MMseqs2 alignment identity to confirm the effect is not an artifact of our k -mer edge metric.

We differ from prior genomics work in what we measure. Those studies document inflated absolute scores within a single leaky domain; we audit a whole suite with a within-study clean control and ask the downstream question—whether leakage reorders *which model wins*. We also position our audit against the deep-model literature it is often compared to: DNABERT-2 (Zhou et al., 2023), HyenaDNA (Nguyen et al., 2023), and the Nucleotide Transformer (Dalla-Torre et al., 2024) are pretrained on the same human reference genome from which these datasets are drawn, so evaluating them here would introduce a second, pretraining-based leakage channel that no train/test split can close (Section 5); we therefore use a from-scratch CNN as the deep-model probe and scope the pretrained foundation models out of the split-effect evidence.

3. Methods

3.1. Datasets

We study the seven binary datasets of the Genomic Benchmarks suite (Grešová et al., 2023), and additionally report the three-class *human_ensembl_regulatory* set separately. Per-dataset sizes, sequence lengths, and class balance are given in Table 1. Sizes range from 6,914 (*drosophila_enhancers_stark*) to 174,756 (*human_ocr_ensembl*) sequences; lengths are fixed within some datasets (e.g. 251 bp for *human_nontata_promoters*) and variable in others; classes are balanced (0.50 positive fraction) except *human_nontata_promoters* (0.544). All balanced datasets are curated to 0.500 in both train and test (positive/negative subfolders), so class balance is preprocessed, not natural—which is why we also run an imbalanced-prevalence stress test (§3.9).

3.2. Features, models, and decision rule

Each sequence is represented by overlapping k -mer counts for $k \in \{4, 6\}$ (4^k features). We evaluate four classifiers spanning a range of *memorization propensity* (equivalently, effective regularization) rather than raw capacity: logistic regression ($C=1.0$, $\text{max_iter}=5000$), a linear support-vector machine (LinearSVC, $C=1.0$, $\text{dual}=\text{False}$), a random forest (150 trees, scikit-learn defaults $\text{min_samples_leaf}=1$, $\text{max_depth}=\text{None}$), and gradient-boosted trees (HistGradientBoosting, library defaults). L2 normalization is applied *only* to the two linear models, via a Normalizer pipeline step; the tree ensembles (random forest, HistGradientBoosting) consume raw counts, for which monotone feature scaling is irrelevant. We deliberately avoid a “capacity” framing: the nominally higher-capacity

Table 1. Datasets in the Genomic Benchmarks binary suite (and the optional three-class set). Lengths are the median (with min–max range where variable); class balance is the positive-class fraction (binary). The 2 bp minimum in *human.enhancers.ensembl* caps its length-pair k -mer Jaccard at ≈ 0.004 , so its Jaccard leak fraction is a severe lower bound (see the containment column of Table 2).

Dataset	n (full)	Length (bp)	Class balance	Test fraction
human.nontata.promoters	36,131	251	0.544	0.25
human.enhancers.ensembl	154,842	269 (2–573)	0.500	0.20
human.ocr.ensembl	174,756	315 (71–593)	0.500	0.20
human.enhancers.cohn	27,791	500	0.500	0.25
demo.coding.vs.intergenomic.seqs	100,000	200	0.500	0.25
demo.human.or.worm	100,000	200	0.500	0.25
drosophila.enhancers.stark	6,914	2142 (236–3237)	0.500	0.25
human.ensembl.regulatory (3-class)	289,061	401 (89–802)	3 classes	0.20

HistGradientBoosting is barely inflated while the random forest is heavily inflated, so the controlled variable is memorization propensity, not capacity. Unless noted, all accuracies use a single decision rule—`.predict()` (`argmax` of the class probabilities)—and the headline accuracy drop is the difference between the original accuracy and the mean corrected accuracy over five re-split seeds, with a percentile test-set bootstrap 95% CI (1,000 resamples of the per-example correctness vector; models are not refit). To confirm strand-symmetry is not the driver, we also re-run the full comparison with canonical k -mer features, `min( $k$ -mer, reverse complement)`.

3.3. Expanded nine-model roster

To test whether the ranking result depends on the size or composition of the comparison set, we widen it from four learners to nine, chosen to span memorization propensity end to end rather than to span accuracy. The four already described are reused *verbatim*—same objects, same normalization, same seeds—and five are added: 1- and 15-nearest neighbours, extremely randomized trees (150 estimators, defaults), a multilayer perceptron (one hidden layer of 128 units, early stopping, at most 60 iterations), and Gaussian naive Bayes. Nearest neighbours is included because it is the *definitional* memorizer—on a near-duplicate its prediction simply is the training label—so it bounds the axis from above; naive Bayes bounds it from below. Scale-sensitive learners (nearest neighbours, the perceptron, and the two linear models) receive the same L2 normalization; tree ensembles are scale-invariant and Gaussian naive Bayes standardizes internally, so both take raw counts. All nine are trained from scratch on each split, so none carries the pretraining-overlap confound that scopes foundation models out of this study, and all use the same $k = 6$ features and the same `.predict()` decision rule.

Two conventions matter for reading the result. First, an inversion criterion designed for four models is too permissive for nine: with a wider roster the top two are often within noise, and “the top model changed by more than a point” fires on a *clean* control from repartitioning alone. We therefore additionally require that the two models exchanging the top slot each lead on their own split by a *paired* cluster-bootstrap difference whose interval excludes zero. Second, we report the margin between the two models that swap and, separately, the margin between the as-shipped first and second place, because those are different quantities and the latter can be a statistical tie even when the former is large. Four predictions were committed in the

modules before they were run, and every one is scored in the Results whether or not it held: **P1** 1-nearest-neighbour shows the largest graded memorization gap; **P2** it shows the largest split-induced drop; **P3** the four memorization-prone learners outrank the two linear models on the as-shipped split and fall below them after correction; **P4** on a clean control no model’s drop interval excludes zero. A fifth, **P5**, was committed in the tuning module of §3.9: that cluster-grouped cross-validation would select a more regularized forest than naive cross-validation. Of the five, P1 holds under the corrected gap and fails under the raw one, P2 holds on one leaky dataset and fails on the other, P3 and P4 hold, and P5 fails outright.

3.4. Near-duplicate measurement

We measure train/test overlap by the exact Jaccard similarity of k -mer sets ($k = 8$), computed for all test–train pairs via sparse binary matrix products (no MinHash approximation). For each test sequence we record its maximum similarity to any training sequence, and define the leakage fraction as the proportion of test sequences whose maximum similarity exceeds a threshold (0.5, 0.7, 0.9). Because a k -mer-set Jaccard is bounded by the length ratio ($\text{Jaccard} \leq L_{\min}/L_{\max}$), it undercounts leakage on datasets with variable or disparate lengths; we therefore also report a length-robust *containment* index, $|A \cap B|/\min(|A|, |B|)$, in the same clustering, and disclose the length-pair cap per dataset (Table 2). Jaccard leak fractions are thus lower bounds; the containment re-measure is the length-robust check on the clean verdicts. We additionally census exact byte-identical train/test duplicates: on *human.enhancers.ensembl*, 11,774 test sequences (0.380 of the test set) are byte-identical training copies (leakage is literal duplication), whereas *human.nontata.promoters* has 0 exact duplicates yet 0.225 near-duplicates at Jaccard ≥ 0.9 (genuinely mutated copies)—the more homology-like but statistically weaker case.

3.5. Near-duplicate-aware re-split

To correct leakage we build a similarity graph in which sequences are connected when their k -mer Jaccard exceeds 0.7, take connected components as near-duplicate clusters, and assign whole clusters to either train or test. The assignment preserves the original train/test ratio and class balance and, by construction, guarantees zero residual cross-split similarity above the threshold (verified empirically). We confirm robustness to the clustering threshold (sweeping 0.5/0.7/0.9)

and to removal of the single largest component. This is a dependency-free instance of the whole-cluster partitioning used by GraphPart (Teufel et al., 2023), CD-HIT (Li and Godzik, 2006), and MMseqs2 (Steinegger and Söding, 2017), any of which could substitute for the edge metric.

3.6. Controls and auxiliary diagnostics

Alignment-based validation.

To test whether the effect depends on the k -mer edge metric, we re-cluster the two leaky datasets with MMseqs2 (Steinegger and Söding, 2017) `easy-cluster` (`min-seq-id 0.7, -c 0.8`), feed the external cluster vector into the *same* whole-cluster assignment path, and refit all four models—only the edge metric changes (alignment identity vs 8-mer Jaccard), the linkage held fixed. Because MMseqs2 is itself k -mer-seeded, we describe this as an *alignment-scored* cross-check, not a k -mer-independent one.

Leave-one-chromosome-out control.

We recover source genomic coordinates for the two leaky datasets and build a chromosome-holdout split—whole chromosomes are assigned entirely to the test side until the target test fraction is reached (nine chromosomes for *human_enhancers_ensembl*, twelve for *human_nontata_promoters*)—the field-standard control in regulatory genomics, refitting all four models. This control is deployment-relevant but *not* leakage-free: residual near-duplicate leakage surviving the chromosome split is 0.8% (*human_enhancers_ensembl*) and 0.02% (*human_nontata_promoters*) at Jaccard 0.7 (versus the exact 0 guaranteed by the near-duplicate-aware split), which we report rather than hide.

3.7. Regularization path and the graded memorization gap

To test whether the effect is an artifact of unregularized random-forest defaults, we sweep `min_samples_leaf` $\in \{1, 5, 20, 50, 100, 200, 500\}$ and `max_depth` $\in \{\text{None}, 16, 8, 4\}$ at full scale (the largest settings exceed the largest near-duplicate cluster, so no single leaf can be a pure duplicate block). The split-internal *graded memorization gap* is $\text{acc}[\geq 0.9] - \text{acc}[\leq 0.5]$: accuracy on near-duplicate test sequences minus accuracy on novel ones. In its raw form this statistic is **not** confound-free, and we correct it. The two similarity strata have very different class composition, in opposite directions on the two leaky datasets—the ≥ 0.9 stratum is 99.97% positive on *human_enhancers_ensembl* against 19.68% in the novel stratum, and 0.15% against 98.17% on *human_nontata_promoters*. A constant classifier that always predicts the positive class, and therefore memorizes nothing, scores a raw gap of +0.803 on the first dataset and −0.980 on the second. Any model with a class-prediction bias inherits a large gap of the corresponding sign for free. We therefore report the gap computed from *balanced* accuracy—the unweighted mean of per-class recall—within each stratum, for which a constant classifier scores 0.5 in every stratum and hence a corrected gap of exactly zero, and we report the raw gap alongside it so the size of the confound is visible. A

label-concordance analysis records whether each near-duplicate test sequence shares its nearest training neighbour's label.

3.8. From-scratch 1D CNN grid

As a deep-model probe we train a from-scratch 1D residual convolutional network (Basset/DeepSEA-style; one-hot input, no k -mer features) on each dataset's training split only, so there is no pretraining confound. We pre-specified, in a timestamped document included in the code release (`results/deep_preregistration.md`), the narrowed claim, the dropout×weight-decay grid, and an explicit refutation condition: if the standard-practice regularized cell (dropout 0.3, weight decay 10^{-4}) has a cluster-bootstrap homology-drop CI including 0 on *both* datasets, the strong “any expressive learner is inflated” reading is refuted and reported as a limitation. Models are deterministic (seed 0), trained with early stopping on a held-out slice, on MPS (CPU fallback); the unregularized-to-convergence cell is a labelled manipulation check.

3.9. Cluster (block) bootstrap

Because near-duplicates violate the independence a sample-wise bootstrap assumes, we also resample whole within-test near-duplicate components (8-mer Jaccard > 0.7 connected components) as blocks. We report whether any significance verdict changes, quantify the design effect via the one-way intraclass correlation (ICC), cross-check the block-bootstrap interval against an analytic Liang–Zeger cluster-robust standard error, and report a combined-source interval that folds the five re-split-seed variance into the test-resampling variance.

Imbalanced evaluation.

Because the balanced datasets are curated, we add a prevalence stress test (random forest, target positive fraction $\pi \in \{0.5, 0.2, 0.1\}$) scored with prevalence-aware metrics: AUPRC (against the prevalence baseline), MCC, balanced accuracy, AUROC, and minority-class precision/recall/ F_1 . The near-duplicate-aware splitter is extended with a per-class realized-fraction check so it preserves the target prevalence.

Tuning selection.

To test whether standard model selection rescues a practitioner from the defect, we cross-validate `min_samples_leaf` $\in \{1, 5, 20, 50, 100, 200, 500\}$ for the random forest on the as-shipped *training* set, under two schemes: ordinary stratified 5-fold, and 5-fold in which whole near-duplicate clusters (the same components used for the corrected split) are held out together. The first is what a practitioner would actually run; the second is the honest version of it. We report which leaf size each selects and what that selection then scores on both splits.

Injected-leakage multiclass construction.

The only multiclass set (*human_ensembl_regulatory*) is clean, so no naturally leaky multiclass data exist in the suite; we answer the multiclass question by construction, injecting label-carrying near-duplicate train→test copies at fraction $f \in \{0, 0.1, 0.2, 0.4\}$ (on a 20k subsample; the mechanism is scale-independent) and comparing the random forest against logistic regression before and after the near-duplicate-aware split.

Definitions used in the robustness results.

Four quantities appear in the robustness results and are defined here. *Cluster cohesion* is the edge density of a single-linkage near-duplicate component: the fraction of its $\binom{m}{2}$ member pairs that actually exceed the similarity threshold. A component built from genuine mutual near-duplicates has cohesion near 1; one assembled by transitive chaining, where members are linked only through intermediates, has cohesion near 0, and de-duplicating it removes sequences that are not in fact near-duplicates of one another. The *design effect* is $1 + (\bar{m} - 1)\rho$ for mean block size $\bar{m}$ and intraclass correlation ρ , i.e. the factor by which correlated blocks inflate the variance of a mean relative to independent sampling; it is what makes a cluster bootstrap matter, or not. A *novel-only* evaluation scores the model trained on the as-shipped split using only test sequences below 0.5 similarity to training, which isolates generalization without retraining or re-splitting. Finally, the *GC-in-distribution* restriction and the low-complexity check use dustmasker (Morgulis et al., 2006) to report the masked fraction of each sequence, distinguishing genuine locus duplication from repeat- or microsatellite-driven similarity.

3.10. Coordinate-space construction signatures

Genomic Benchmarks distributes per-sequence intervals (id,region,start,end,strand) separately from the sequences its loader returns. We recover them offline and realign them to loader row order, which is deterministic (split, then sorted class directory, then sorted filename, the filename stem being the interval id). Recovered interval width equals sequence length for 100% of rows in all eight datasets; that check is informative only for the four variable-length datasets, since it is satisfied trivially where every sequence has the same length. Because these coordinates are causally upstream of, and statistically independent from, the k -mer clustering that measures leakage, they provide a cross-modal test rather than a restatement. Per class we compute self-overlap redundancy $R_{\text{self}} = 1 - n_{\text{merged}}/n_{\text{raw}}$ and the share of *test* intervals overlapping any *train* interval at $\geq t$ reciprocal overlap, $t \in \{\geq 1 \text{ bp}, 0.5, 0.9, 1.0\}$. Overlap is strand-agnostic, since strand is unrecorded for two datasets. The screen thresholds the maximum over classes of the $\geq 50\%$ reciprocal cross-split statistic at 0.1—the same cut already used for the sequence-space verdict—so it introduces no fitted parameter.

3.11. Construct-and-break manipulations

To test the construction account causally rather than correlationally we intervene on the construction step alone. *Break-a-clean*: *human.ocr.ensembl*, drawn from the merged Ensembl Regulatory Build and clean by every measure, is rebuilt the way an un-duplicated assembly is, by duplicating positives until the same 94.3% share of positive rows sits on a multiplicity-2 coordinate as we measure on *human.enhancers.ensembl* (which requires duplicating 89.1% of them, since duplicating a fraction r leaves $2r/(1+r)$ of rows on a duplicated coordinate), then downsampling to the control's exact size and class balance. *Fix-a-leaky*: *human.enhancers.ensembl* receives the merge step its curators omitted—every duplicated coordinate collapsed to one representative row—with sequences, labels, model code

and split ratio unchanged and negatives subsampled to restore the control's balance. Each manipulation is paired with an unmanipulated control at the same split seed. Only the positive class carries duplicated coordinates, so both interventions perturb balance and size unless corrected; we therefore report balance-matched arms as primary and disclose that the break-a-clean pair is matched on size as well while the fix-a-leaky pair is not, since size there cannot be equalised without discarding the rows the intervention is defined to remove.

3.12. Cross-suite census and re-ranking

We apply the census of §3.4 unchanged to the Nucleotide Transformer downstream tasks (Dalla-Torre et al., 2024), then run the full audit pipeline—as-shipped split, near-duplicate-aware re-split, same four models, same decision rule—on the tasks it flags, with a clean task as control. One deviation must be declared, because it affects the task that anchors the cross-suite result. The *enhancers* task ships only 400 test sequences, too few to carry a usable interval, so for that task alone we pool train and test and draw a fresh stratified split at the same ratio before running both arms. Pooling redistributes the duplicate pairs, so the leak fraction we measure on the re-split ($\varphi = 0.099$) is lower than the 25% byte-identical figure the as-shipped split carries; the census number describes the release as shipped, the ranking number describes the split we actually trained on, and they are not the same quantity. Training sets are capped at 20,000 sequences for the census; capping can only remove potential near-duplicate partners, so every reported fraction is a lower bound. Two counting conventions matter. *enhancers* and *enhancers.types* share identical sequence sets and differ only in label granularity, so they are one task scored twice and we report independent-task counts. And although the suite ships original and revised releases, we draw no before-and-after inference from the pair: they are not the same data recurred—the original splice tasks are keyed by UniProt accession across species while the revised are human genomic windows keyed by coordinate, and sequence length differs on eight of thirteen tasks—so each release is assessed on its own terms.

3.13. A diagnostic condition for when leakage reorders

Stratify a leaky test set by maximum similarity to training into a leaked stratum (similarity ≥ 0.9 , fraction φ) and a novel stratum (< 0.5), and write h_M and n_M for a model's accuracy in each and $g_M = h_M - n_M$ for the graded gap of §3.7. Then

$$a_M \approx n_M + \varphi g_M, \quad (1)$$

and for an ordered pair (A, B) with A leading the as-shipped split, writing $\delta = n_B - n_A$ and $\Delta g = g_A - g_B$, the as-shipped margin is $\varphi \Delta g - \delta$, so the benchmark reports the wrong winner when

$$0 < \delta < \varphi \Delta g, \quad \text{equivalently } \varphi > \varphi^* = \delta / \Delta g. \quad (2)$$

Two honest qualifications, because (1) is easy to overstate. First, it is an *approximation and not an identity*: the two strata do not partition the test set, since sequences in the

intermediate $[0.5, 0.9)$ band belong to neither. That band is negligible on *human.enhancers.ensembl* (1.4% of the test set, reconstruction error 0.002) but not elsewhere—22.4% on *human.nontata.promoters* (error up to 0.031) and about a third on the Nucleotide Transformer splice tasks, where the approximation error exceeds the margins it would be used to adjudicate. We therefore apply (2) only where the intermediate band is small, and report the band's size wherever we use it. Second, (2) is bookkeeping rather than theory—two strata and an ordering—so we report it as a diagnostic screen, valuable only because φ , n and g are all measurable on the as-shipped split, letting a curator evaluate the condition *without building a corrected split*. It is tested out of sample in §4.10 and is not offered as a contribution in its own right.

3.14. Reproducibility

All classical experiments are deterministic with logged seeds, CPU-only, and free of external services; the deep-model runs are seeded and single-device. The near-duplicate-aware splitter is released as a standalone tool (`homology.split.py`, dependencies `numpy/scipy`). Reported 95% confidence intervals are percentile bootstrap intervals; the resampling unit is the individual test sequence (sample bootstrap) or the within-test near-duplicate component (cluster bootstrap), models are not refit, and seeds are fixed.

4. Results

4.1. Leakage is dataset-specific and clean verdicts survive a length-robust re-measure

Of the seven binary datasets, only two—*human.enhancers.ensembl* and *human.nontata.promoters*—exceed a 10% near-duplicate test fraction at Jaccard 0.7 (0.384 and 0.406 respectively); the remaining five are near zero, and the three-class *human.ensembl.regulatory* set is likewise clean (0.005; Table 2). Because a k -mer Jaccard is length-capped, we re-measured at full scale with the length-robust containment index. Containment reveals substantially more leakage on *human.enhancers.ensembl* (0.384 $\rightarrow$ 0.845; its 2bp minimum length caps the length-pair Jaccard at $\approx$ 0.004), and it flags one dataset the length-blind metric misses: *demo.coding-vs.intergenomic.seqs*, clean under Jaccard (0.078), crosses the threshold under full-scale containment (0.131), so we relabel it *borderline*. The remaining four clean datasets stay well below the threshold under both metrics (largest full-scale containment 0.068, *human.ocr.ensembl*). Two datasets are thus clearly leaky, one is borderline (containment-only), and four are clean under both metrics; the containment result is itself a demonstration that a length-blind similarity can under-report leakage on variable-length data, so both metrics should be reported.

Negative control: only the near-duplicate-aware split removes the inflation

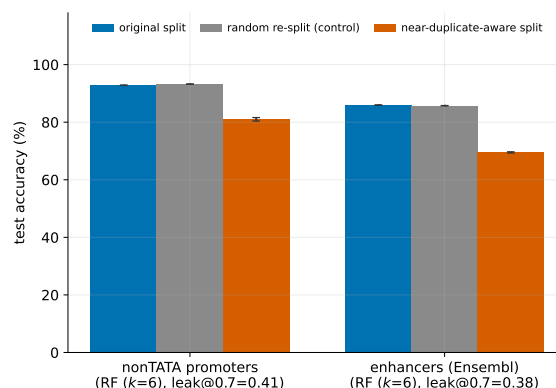


Figure 1 Original split versus random re-split (negative control) versus near-duplicate-aware re-split, best model per dataset. Only the near-duplicate-aware split lowers accuracy, and only on leaky datasets. The y -axis starts at 0; best-model accuracies span roughly 70–93%.

4.2. Correcting the split lowers accuracy only on leaky datasets

Under the near-duplicate-aware re-split, the best model's accuracy drops sharply on the leaky datasets—by 16.5 points for the random forest on *human.enhancers.ensembl* ($0.8596 \rightarrow 0.6951 \pm 0.0019$ over five re-split seeds; the seed-0 partition used for the intervals and for Table 3 gives 0.6957, a 16.4-point drop; combined-source 95% CI [15.4, 17.4], folding the test-set and five-seed re-split variance) and 11.8 points on *human.nontata.promoters* ($0.9284 \rightarrow 0.8101 \pm 0.0055$ over five re-split seeds). For that second dataset the point estimate and the interval are computed on different estimands and we state both rather than pairing them misleadingly: the five-seed mean gives the 11.8-point figure, while the cluster-bootstrap interval is computed on the seed-0 partition, where the delta is 10.8 points, giving 95% CI [8.8, 12.9]—whereas every clean-dataset delta is null. All five non-leaky deltas include zero for the memorizing random forest as well as the linear models (the largest, *drosophila.enhancers.stark* RF, is $+0.024$ with 95% CI $[-0.008, 0.058]$), so the positive control holds for the model that carries the effect.

The negative control confirms causation: a random re-split of the same data, at matched size and balance, leaves the leaky-dataset accuracies essentially unchanged (best-model deltas -0.002 and -0.001 , both CIs including zero). Only removing near-duplicates, not re-splitting per se, lowers the score (Fig. 1). The corrected accuracy is stable across five re-split seeds and, independently, across five random-forest training seeds (0.698 ± 0.002 on *human.enhancers.ensembl*, 0.821 ± 0.003 on *human.nontata.promoters*), with the random forest demoted on every seed—so the effect is neither a fortunate partition nor model stochasticity.

4.3. Near-duplicate leakage reorders the model ranking on one dataset

The consequence for method comparison is the central result, and it separates the two leaky datasets. On

Table 2. Near-duplicate leakage report card. **Verdict** is LEAKY if the full-scale near-duplicate test fraction exceeds 0.1 at Jaccard 0.7; a “clean” verdict means only *no forward-strand near-duplicate leakage detected*, subject to the disclosed length-pair cap. “leak@0.7 (Jac / con)” gives the leak fraction under the 8-mer Jaccard and the length-robust containment index. “Acc. lost” is the best-model accuracy change under the near-duplicate-aware re-split (positive = accuracy lost; accuracy fraction). “RF rank” is the random-forest rank before→after correction. “Reorders under chrom.-holdout?” reports whether the best-model change reproduces under the chromosome-holdout control (measured for the two leaky datasets only). All leak fractions are full-scale. [†]*demo_coding_vs.intergenomic_seqs* is clean under 8-mer Jaccard (0.078) but its full-scale containment leak fraction (0.131) exceeds 0.1—a borderline case the length-robust metric flags and the length-blind one misses.

Dataset	<i>n</i> (full)	leak@0.7 (Jac)	leak@0.7 (con)	Verdict	Acc. lost	RF rank	Reorders under chrom.-holdout?
human.nontata.promoters	36,131	0.406	0.445	LEAKY	+0.118	1→3	no (RF stays rank 1)
human.enhancers.ensembl	154,842	0.384	0.845	LEAKY	+0.165	1→4	yes (RF → rank 4)
demo.coding_vs.intergenomic_seqs	100,000	0.078	0.131	borderline [†]	−0.001	4→4	n/a
human.ocr.ensembl	174,756	0.010	0.068	clean	+0.004	4→4	n/a
demo.human_or_worm	100,000	0.012	0.042	clean	−0.004	4→4	n/a
drosophila.enhancers.stark	6,914	0.016	0.033	clean	+0.012	2→3	n/a
human.enhancers.cohn	27,791	0.001	0.005	clean	−0.004	4→4	n/a
human.ensembl.regulatory (3-class)	289,061	0.005	–	clean	−0.010	n/a	n/a

human.enhancers.ensembl the best model *changes*: the default random forest, ranked first on the standard split (accuracy 0.860), falls to last after correction (accuracy 0.696), and *LinearSVC* becomes best (0.798; corrected order *LinearSVC*>*LR*>*HGB*>*RF*; Fig. 2).

This change holds along every axis we tested. (i) It holds under all three metrics—the random forest is demoted to rank 4 under accuracy, AUROC (0.805 vs *LinearSVC* 0.870 corrected), and F_1 (0.635 vs 0.796)—so it is not a threshold artifact. (ii) The bootstrap probability that the random forest ranks first goes from $P = 1.0$ on the standard split to $P = 0.0$ after correction, with *LinearSVC* at $P = 1.0$ (both sample- and cluster-bootstrap). (iii) It reproduces under a chromosome-holdout control (§4.11).

The size of the forest’s fall repays a closer look, because it decomposes into two distinct effects and only one of them is the split. Evaluated on the as-shipped split’s *novel* stratum—no re-splitting, no retraining—the forest scores 0.770 against *LinearSVC*’s 0.782: the ranking has already changed, but by only 1.2 points. After re-splitting and refitting it sits 10 points below. The difference is a *retraining penalty*: de-leaking removes near-duplicate partners from the training set as well as the test set, and the memorizer loses more from that than the other models do (writing a^{corr} for accuracy on the corrected split and n for accuracy on the as-shipped split’s *novel* stratum, the residual $a^{\text{corr}} - n$ is −0.074 for the forest against +0.012 to +0.016 for the other three). Both components are real and both are consequences of the leakage, but they are different quantities: the first says the benchmark was ranking the wrong model, the second says the memorizer’s advantage depended on training data it should not have had.

On *human.nontata.promoters* the picture is weaker. Under accuracy at threshold 0.5 the random forest is demoted from rank 1 to rank 3 (corrected *LR* 0.829 > *HGB* 0.827 > *RF* 0.820 > *LinearSVC* 0.808; these four accuracies are the single seed-0 partition, used for an apples-to-apples within-split comparison, and the forest’s 0.820 here is the seed-0 value of the same quantity whose five-seed mean, 0.810, gives the headline drop), but this is a statistical dead heat—the corrected accuracy CIs of the leaders overlap (*RF* [0.812, 0.828]

vs *LR* [0.821, 0.837])—and it does not survive a threshold-free metric: under both AUROC and F_1 the random forest remains rank 1 (corrected AUROC 0.929, the highest of the four). The bootstrap winner probability agrees: after correction no model dominates (*LR* 0.63, *HGB* 0.31, *RF* 0.06, cluster-bootstrap). We therefore claim the ranking-change result cleanly for *human.enhancers.ensembl*, as an existence proof, and only partially for *human.nontata.promoters*. Per-panel rank correlations (Fig. 2) are consistent but, with only four models, Kendall τ is too coarse to be a primary statistic—a fall from first to last flips only its own pairs, leaving $\tau \approx 0$ despite being the most consequential change in the study.

4.4. The reordering is not an artifact of a four-model comparison

The obvious objection to a claim about “rankings” is the size and composition of the comparison set. We therefore widened it to nine learners spanning the memorization axis end to end—1- and 15-nearest neighbours, a random forest, extremely randomized trees, a multilayer perceptron, gradient boosting, a linear SVM, logistic regression, and Gaussian naive Bayes—all trained from scratch, on the same splits, with the same decision rule (Table 3). Predictions P1–P4 (§3.3) were committed before the run.

On *human.enhancers.ensembl* the wider roster strengthens the result and removes the “sklearn tree defaults” reading entirely. The corrected winner is the linear SVM, the same model the four-model comparison selects, so the two analyses agree. What the wider roster adds is a sharper statement of the damage. **The leaky benchmark cannot separate its top three at all:** a multilayer perceptron (0.8736), 1-nearest-neighbour (0.8729) and extremely randomized trees (0.8721) finish within 0.0015 of one another, and the paired interval on the top-two margin, [−0.005, 0.006], includes zero—a three-way tie. **On the corrected split those same three models span 23 accuracy points** (0.768, 0.537, 0.628). The benchmark is not merely crowning the wrong model; it is blind to enormous real differences between models it ranks as equals.

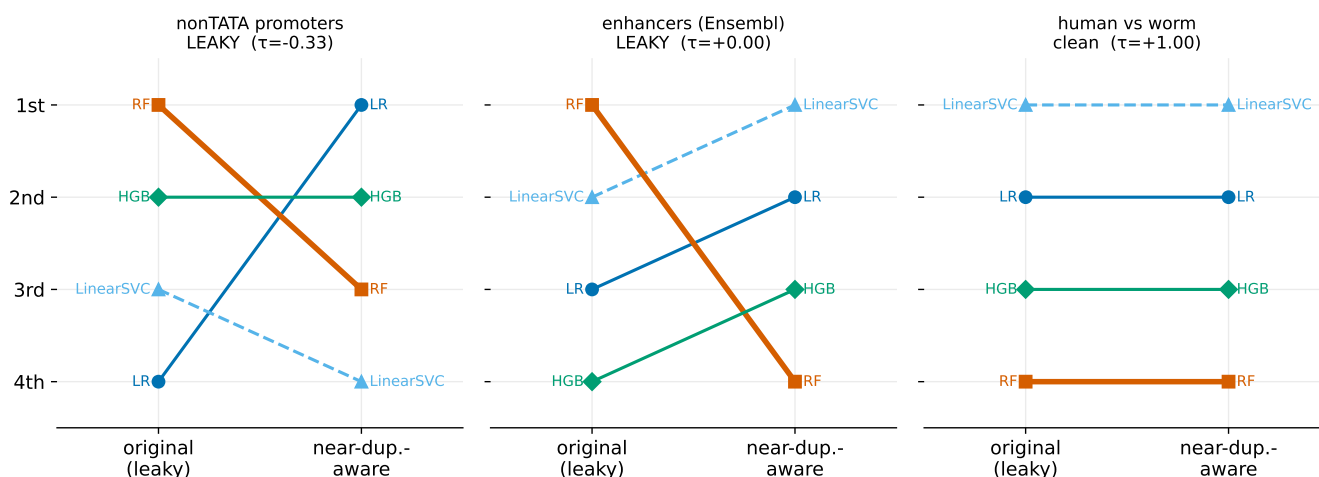
Model ranking: original (leaky) vs near-duplicate-aware split ($k=6$, accuracy)

Figure 2 Model rankings on the original (leaky) split versus the near-duplicate-aware split ($k=6$, accuracy). The random forest is dethroned on *human.enhancers.ensembl* (middle) and, under accuracy only, on *human.nontata.promoters* (left), but rankings are stable on a clean control (right). Per-panel τ is the rank correlation; on *human.enhancers.ensembl* the random forest's move to last flips only its own pairs, giving $\tau \approx 0$ despite the dramatic demotion—illustrating why τ is too coarse to serve as a primary statistic with only four models (§4.3).

The reordering itself is decisive and supported. The linear SVM rises from rank 5 to rank 1 and logistic regression from 6 to 2, while the perceptron falls from 1 to 3, extremely randomized trees from 3 to 7, and 1-nearest-neighbour from rank 2 to rank 9, last. The two models that exchange the top slot each lead on their own split by a margin whose paired cluster-bootstrap interval excludes zero ($[0.071, 0.080]$ as shipped, $[0.024, 0.035]$ after correction). The split between models is the one the memorization account predicts: **the four models whose drops exclude zero are 1-NN, the forest, extremely randomized trees and the perceptron; the remaining five—including both linear models, boosting and naive Bayes—are all null.** The perceptron's presence in that group is worth stating plainly rather than filing it under “memorizers”: it is not one of the four learners we designated memorization-prone in advance, and its inclusion says the inflation reaches a neural learner too. The clearest single number is 1-NN, which scores 0.873 on the as-shipped split and 0.537—indistinguishable from chance—once its near-duplicate partners are removed.

The convolutional network belongs in this comparison, and we report it with the caution its evidence deserves rather than the emphasis its conclusion would invite. Its pre-registered reference cell is built on the same corrected split by the same construction, and it scores 0.8272 as shipped—fifth, behind the perceptron, 1-NN, extremely randomized trees and the forest—and 0.8131 after correction, which would place it first. But that single cell is the *maximum* of the nine-cell regularization grid, whose corrected accuracies span 0.749–0.813, and the linear SVM's corrected 0.7975 falls *inside* that range: five of the nine cells exceed it and four fall below. We therefore do not claim the network would win the corrected ranking, and we would be violating both our own inversion criterion (§3.3) and our own instruction not to read conclusions from individual grid cells (§4.9) if we did. What the comparison supports is

weaker and still worth stating: the network is ranked fifth by the leaky split while its corrected accuracy is comparable to or better than that of models ranked above it, so the leaky split is at least not *favouring* the architecture family this suite publishes baselines for—and a properly powered version of this comparison, with seed replication and an interval, is the single most useful experiment a follow-up could run.

Of P1 and P2 on this dataset, the second holds outright—1-NN has the largest drop (0.336)—and the first depends on which form of the gap is used, which is itself informative. Under the *raw* gap it fails, the forest edging 1-NN 0.230 to 0.208; under the prevalence-corrected gap it holds decisively, 1-NN leading at +0.461 against the forest's +0.319 (§4.5). We record both rather than quoting whichever is convenient: the prediction was made about memorization propensity, and the corrected statistic is the one that measures it. On *human.nontata.promoters* both fail—15-NN has the largest gap and extremely randomized trees the largest drop—and we report that rather than the *ensembl* outcome alone, since a pre-registration reported selectively is worth nothing. P3, that memorizers outrank the linear models before correction and fall below them after, holds decisively on *human.enhancers.ensembl*: mean rank 4.50 against 5.50 as shipped, 7.25 against 1.50 after. P4 holds: on the clean control no model's drop interval excludes zero, for any of the nine.

On *human.nontata.promoters* the roster produces an even larger scramble—extremely randomized trees fall from rank 1 to 8 and 1-NN rises from 3 to 1, with supported margins—but we decline to read it as strengthening that dataset's case. The reason is the prevalence-corrected graded gap of §4.5: on that dataset it does not separate the model families at all (memorizers +0.122 against +0.166 for the rest, with the largest belonging to the perceptron), so the movement is not attributable to near-duplicate memorization even though it is large. That agrees with the independent evidence that this dataset's clusters chain heavily and its

Table 3. Nine-model roster on *human_enhancers_ensembl*, full scale, as-shipped versus near-duplicate-aware split. Gap_{bal} is the graded memorization gap computed from *balanced* accuracy within each stratum and is the statistic we rely on; Gap_{raw} is the uncorrected accuracy difference, shown only for comparison because it is confounded by stratum class composition (§3.7)—a constant classifier scores +0.803 on this dataset, which is why the raw column contains two large negative values that do not indicate “negative memorization”. Note that the corrected column orders the models as the memorization account predicts and the raw column does not. “Drop” is the accuracy lost under correction, with a two-arm cluster-bootstrap interval; * marks intervals excluding zero. Models are ordered by as-shipped rank. The top three as-shipped finish within 0.0015 of one another—a tie—yet span 23 points on the corrected split. The corrected winner is the linear SVM, agreeing with the four-model comparison of §4.3.

Model	Gap_{bal}	Gap_{raw}	As-shipped	Corrected	Drop	Rank
MLP	0.234	0.159	0.8736	0.7684	0.105*	1 → 3
kNN-1	0.461	0.208	0.8729	0.5370	0.336*	2 → 9
ExtraTrees	0.370	0.210	0.8721	0.6278	0.244*	3 → 7
RandomForest	0.319	0.230	0.8596	0.6957	0.164*	4 → 5
LinearSVC	0.144	0.037	0.7980	0.7975	0.001	5 → 1
LogisticReg.	0.154	0.037	0.7809	0.7806	0.000	6 → 2
HistGB	0.162	0.034	0.7636	0.7608	0.003	7 → 4
GaussianNB	0.129	−0.234	0.6482	0.6492	−0.001	8 → 6
kNN-15	0.016	−0.684	0.5357	0.5371	−0.001	9 → 8

corrected split over-removes genuine promoter signal. We note explicitly that a tempting alternative discriminator does *not* work: the residual of each drop against φg is in fact *larger* on *human_enhancers_ensembl* (up to +0.259) than on *human_nontata_promoters* (up to +0.152), so it cannot be used to prefer one dataset over the other—it measures the train-side retraining penalty of §4.3, which is a consequence of leakage on both. The wider roster therefore sharpens the existing verdict rather than overturning it: *human_enhancers_ensembl* remains the clean case and *human_nontata_promoters* the cautionary one.

4.5. The mechanism is memorization of label-carrying near-duplicates

Binning test sequences by similarity to training reveals the mechanism directly (Fig. 3). On *human_enhancers_ensembl*, the default random forest scores 1.000 on near-duplicate test sequences (≥ 0.9 similarity) but only 0.770 on novel ones (< 0.5)—a graded gap of +0.230, versus ≈ 0.035 for the other models—and on the novel sequences the random forest (0.770) is actually *worse* than the linear SVM (0.782).

That raw comparison, however, is confounded by class composition (§3.7), and correcting it is worth doing because the correction *strengthens* the mechanism rather than weakening it. Recomputing every gap from balanced accuracy within each stratum—so that a constant classifier scores exactly zero—orders the nine-model roster on *human_enhancers_ensembl* as the memorization account predicts and the raw statistic did not: 1-nearest-neighbour, the definitional memorizer, now has by far the largest corrected gap (+0.461), followed by extremely

randomized trees (+0.370) and the forest (+0.319), with the linear models at the bottom (+0.144, +0.139). The memorizers average +0.292 against +0.147 for the rest. The correction also disposes of two values the raw statistic produced that were never interpretable as memorization at all: 15-nearest-neighbour’s raw −0.684 and naive Bayes’s −0.234 become +0.016 and +0.141, i.e. artifacts of predicting the minority class in a stratum that is 99.97% positive.

The same correction is equally informative where it removes a signal. On *human_nontata_promoters* the corrected gaps do *not* separate the families—memorizers average +0.122 against +0.166 for the others, and the largest belongs to the perceptron—so on that dataset the mechanism is not supported once class composition is accounted for. This is an independent statistic reaching the same clean-versus-cautionary split that §4.3, the chromosome control and the cohesion analysis reach separately, and we take the convergence as the reason to scope the mechanism claim to *human_enhancers_ensembl*.

Memorization pays because near-duplicates carry the label: at ≥ 0.9 similarity, 99.99% of near-duplicate test sequences share their nearest training neighbour’s label on *human_enhancers_ensembl* ($n = 11,774$) and 99.95% on *human_nontata_promoters* ($n = 2,037$), so recognizing a near-copy is correct essentially for free. On *human_enhancers_ensembl* the inflation is confined to the memorizing model: the random-forest drop is significant while the logistic-regression drop is null (+0.0004, 95% CI [−0.006, +0.007]). Clean datasets have near-empty high-similarity bins, leaving nothing to memorize.

4.6. The cause is a curation defect visible in the coordinates

The suite ships genomic intervals alongside its sequences, and they identify the defect directly. Of the 77,421 positive intervals in *human_enhancers_ensembl*, only 40,934 are distinct: 36,487 coordinates are shipped exactly twice and 4,447 once, and every one of the 36,487 duplicated pairs carries byte-identical sequences and a single label, so the coordinate redundancy is the sequence redundancy. Because the shipped split is random, 75.5% of test positives have their exact coordinate present in training.

The comparison that most nearly isolates the cause is internal to Ensembl, and we state its limits before drawing on it. *human_enhancers_ensembl*’s positive class derives from the FANTOM5 CAGE enhancer atlas (Andersson et al., 2014), whereas *human_ocr_ensembl* and *human_ensembl_regulatory* derive from the Ensembl Regulatory Build’s merged “MultiCell” segmentation (Zerbino et al., 2015). Provider, organism and assembly are held fixed to the extent that all three are human regulatory element sets distributed through Ensembl, but the upstream resources differ, so this is *not* one pipeline run with and without a merge step and we do not claim it is. What the contrast establishes is an association between the presence of a deduplication step and the absence of the defect; the causal claim rests on the manipulations of §4.7, where the construction step is varied directly and everything else is held fixed by construction. *human_ocr_ensembl* and *human_ensembl_regulatory* are drawn from the merged “MultiCell” Regulatory Build and sit at 0.000–0.010 self-overlap,

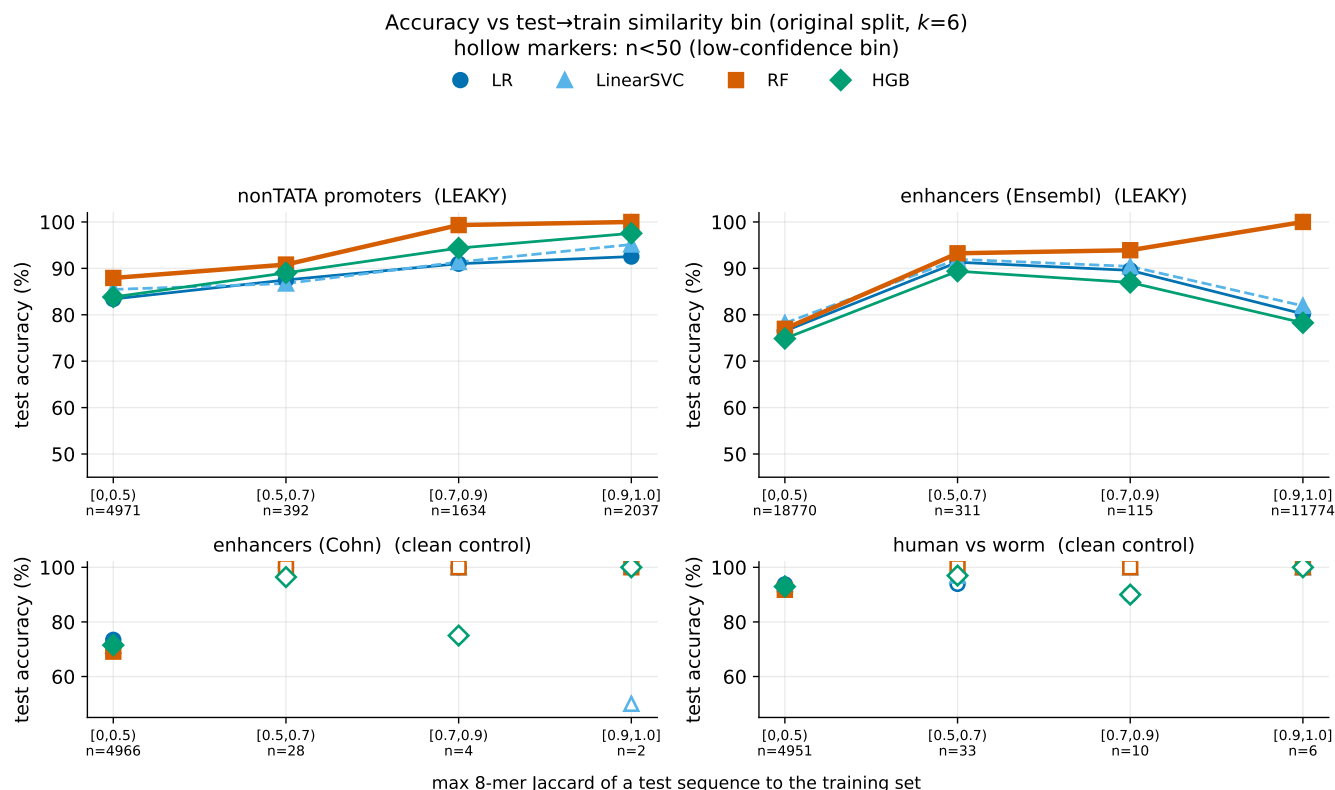


Figure 3 Accuracy as a function of the test-to-train similarity bin (original split, $k=6$). On leaky datasets the default random forest is near-perfect on high-similarity (near-duplicate) test sequences and falls on dissimilar ones; clean controls have near-empty high-similarity bins (hollow markers: $n < 50$).

whereas the un-deduplicated *human_enhancers_ensembl* sits at 0.471. Thresholding the maximum-over-classes cross-split coordinate overlap (§3.10) at the same 0.1 cut used in sequence space separates the suite 7/8, with the two leaky datasets at 0.755 and 0.992 against 0.073 and below for the rest. The single miss is *demo_coding_vs_intergenic_seqs*, containment-borderline in sequence space but coordinate-clean, whose leakage is coding-paralog homology with no coordinate signature by construction—and whose *coding_seqs* class is keyed by transcript accession, one region per row, so its coordinate statistics are structurally zero rather than measured. Coordinates may therefore escalate a verdict but must never clear one. Two further refinements were forced by the data: redundancy is not always in the positive class (*human_nontata_promoters* carries essentially all of its own in the *negative* class, $R_{\text{self}} = 0.961$ against 0.047), and mere adjacency over-calls (*drosophila_enhancers_stark* tiles at 2142 bp with base-pair-scale touches, 0.406 at ≥ 1 bp but 0.036 at $\geq 50\%$ reciprocal overlap).

4.7. Editing the construction step alone creates and cures the reordering

The coordinate evidence is correlational; intervening on the construction step makes it causal, in both directions (§3.11, Table 4). Applying the omitted merge step to *human_enhancers_ensembl*, at matched class balance, collapses its leak fraction from 0.390 to 0.012 and the random-forest

inflation from +0.165 to -0.008 —a null—and the reordering disappears, the forest sitting at rank 4 both before and after correction. Conversely, imposing the same defect on the merged, clean *human_ocr_ensembl*, at identical size and balance, raises its leak fraction from 0.004 to 0.204 and its random-forest inflation from +0.002 to +0.105, and manufactures the reordering: the forest is promoted from rank 4 to rank 1 on the contaminated split and demoted to rank 4 again by correction—the syndrome of §4.3, produced on demand. The break-a-clean effect is larger under matching than without it ($+0.063 \rightarrow +0.105$), so the balance and size confounds were suppressing rather than creating it.

Three caveats belong with this result. Duplicated rows scattered across a random split are leaky by construction, so the break-a-clean direction is unsurprising in itself; its content is that the full downstream syndrome, including the memorizer's promotion to rank 1, follows from the construction step alone. The size-matching downsample removes some of the duplicate partners it has just introduced, so the imposed leak fraction (0.204) is about half the undownsamped value (0.498), making the resulting effect conservative. And each condition is a single fit at a single split seed with no interval attached, so we read the order-of-magnitude contrast against the control and not small differences between arms.

Table 4. Construct-and-break manipulations. Each intervention is paired with an unmanipulated control run of the identical pipeline at the same split seed and matched class balance; the break-a-clean pair is matched on size as well. “Leak” is the near-duplicate test fraction at Jaccard 0.7; “RF drop” is the random-forest accuracy change under the near-duplicate-aware re-split; “RF rank” is before→after correction. Editing coordinates alone abolishes the reordering in a leaky dataset and manufactures it in a clean one.

	Condition	<i>n</i>	Leak	RF drop	RF rank
Fix-a-leaky	<i>ensembl</i> , as shipped duplicates collapsed	154,842	0.390	+0.165	1 → 4
		81,868	0.012	−0.008	4 → 4
Break-a-clean	<i>ocr</i> , as shipped positives duplicated	20,000	0.004	+0.002	4 → 4
		20,000	0.204	+0.105	1 → 4

4.8. The effect is a property of the default unregularized random forest

Regularizing the random forest dissolves the entire phenomenon monotonically. At the default (`min_samples_leaf=1`), the forest scores exactly 1.000 on the ≥ 0.9 -similarity bin (pure leaves form around individual near-duplicates), with the large drop (0.164 on *human.enhancers.ensembl*, 0.108 on *human.nontata.promoters*), the large graded gap (0.230/0.121), and rank 1. Increasing `min_samples_leaf` to 500 drives the homology drop to ≈ 0 (0.164 → −0.001; 0.108 → 0.000), collapses the graded gap (0.230 → 0.020; 0.121 → 0.061), and lowers accuracy on the ≥ 0.9 bin from 1.000 to ~ 0.71 –0.82; capping `max_depth` drives the drop to ≈ 0 likewise. Crucially, the “random forest wins on the leaky split” phenomenon itself *requires* the unregularized forest: on the standard split the forest holds rank 1 only near the default and has fallen to rank 4 by `min_samples_leaf` ≥ 20 on both datasets. Either intervention—heavy regularization or a near-duplicate-aware split—reveals the truth; the leakage inflation and demotion are a property of the default high-capacity forest practitioners deploy.

4.9. Deep models confirm the mechanism

A from-scratch 1D CNN, trained only on each training split, shows the same inflation, so the effect is not a classical-model artifact. The pre-registered standard-practice regularized reference cell (dropout 0.3, weight decay 10^{-4}) drops on *both* datasets—+0.043 on *human.nontata.promoters* (cluster-bootstrap CI [0.025, 0.061]) and +0.014 on *human.enhancers.ensembl* ([0.007, 0.021]), both excluding zero—so the pre-registered refutation condition is *not* triggered. Nineteen of the 20 cells drop with cluster-bootstrap CIs excluding zero (all nine *human.enhancers.ensembl* grid cells, eight of nine *human.nontata.promoters* grid cells, and both manipulation checks); the sole exception is one *human.nontata.promoters* cell (dropout 0.6, weight decay 10^{-4}) where heavy dropout under-fits, leaving no inflation to lose (accuracy rises 0.020 after correction). Per-cell single-run variance is non-trivial, so we read the mechanism from the

grid-level contrast—regularized reference versus unregularized manipulation check—rather than from individual cells.

The unregularized-to-convergence manipulation-check cell memorizes hardest, mirroring the unregularized random forest: on *human.enhancers.ensembl* its graded gap is +0.222 (accuracy 0.974 on the ≥ 0.9 bin vs 0.752 novel) with drop +0.076 [0.068, 0.084]. One pre-registered expectation was not met, and we report it rather than quietly restate the grid: the pre-registration named this a *dose-response* grid, and it is not one. Across the nine cells the drop is non-monotone in both dropout and weight decay (on *human.enhancers.ensembl* it runs 0.021/0.036/0.058, 0.018/0.014/0.064, 0.029/0.063/0.016), so the grid establishes that the inflation reaches a from-scratch neural network but not that it scales smoothly with regularization strength. The one clean contrast is training duration: the unregularized-to-convergence check differs from the matched grid cell only in epochs. This is consistent with “memorization propensity, not capacity”.

4.10. A second suite carries the defect; its rankings do not invert, as the condition predicts

Applying the census unchanged to the Nucleotide Transformer downstream tasks shows the defect is not confined to Genomic Benchmarks. Counting independent tasks, **three of twelve** in the original release are leaky and three more borderline. The enhancer task carries 25.0% test-to-train leakage that is *byte-identical*—verified on a separate code path using no *k*-mers, 100 of 400 test sequences with label concordance 1.000, in a training set holding 14,968 rows on 14,002 unique sequences—while *splice_sites.acceptors* and *donors* carry 0.244 and 0.245 at Jaccard 0.7 by a different mechanism, with zero exact duplicates, both being multi-species sets whose near-duplicates are homologous splice sites across organisms.

We then ran the full pipeline there, and report a null: **no material ranking inversion** on any of the three leaky tasks, nor on the clean control. Leakage inflates scores as expected—on *enhancers* the random forest drops 0.0413 [0.0241, 0.0583], *LinearSVC* 0.0221 [0.0055, 0.0381] and *HistGradientBoosting* 0.0185 [0.0021, 0.0366], all excluding zero—but the leading model is unchanged everywhere.

This also puts condition (2) to its first out-of-sample test, and we are careful about how much that test can show, because most of it is vacuous by construction. Every pairwise prediction was emitted from as-shipped measurements before any corrected split existed, and 24 of 24 pair orders were predicted correctly—but a constant “nothing swaps” predictor scores 23 of 24 on the same data, so the aggregate number is nearly uninformative. What matters is the two pairs where $\delta > 0$ and the condition therefore had something to say: it predicted no swap for *splice_sites.acceptors* RF-versus-HGB ($\varphi = 0.077$ against $\varphi^* = 0.257$) and a swap for *enhancers* RF-versus-HGB ($\varphi = 0.099$ against $\varphi^* = 0.073$). Both were right, and the second is precisely the case the trivial baseline gets wrong: *HistGradientBoosting* does overtake the forest after correction (0.887 versus 0.869). That swap is nonetheless *immaterial* under our own convention, since the two were separated by only 0.005 on the as-shipped split, which is why the task reports

no inversion. So the condition earns two informative calls out of twenty-four opportunities and gets both; that is real but slender evidence, and we do not present it as more.

The reason for the null is the condition's own: $\delta \leq 0$ on twenty-two pairs, because on these tasks the model leading the as-shipped split is, with one exception, also the best model on novel sequences—logistic regression on *enhancers* and on the control task, the forest on *splice_sites_donors*—so there is nothing to invert. The exception is *splice_sites_acceptors*, where boosting is best on novel sequences while the forest leads as shipped; there the condition correctly identified a possible inversion and correctly predicted it would not occur, $\varphi = 0.077$ falling below $\varphi^* = 0.257$. Reordering needs a challenger that is actually better on novel sequences and a leak fraction above φ^* ; this suite supplies the second condition without the first. The reordering result therefore remains a single-dataset existence proof, now externally stress-tested rather than merely asserted.

4.11. Three controls: alignment identity, chromosome holdout, and block resampling

Alignment identity defuses the k -mer-circularity concern.

Re-clustering by MMseqs2 alignment identity, feeding the same whole-cluster splitter, reproduces the *human_enhancers_ensembl* effect metric-independently: the random-forest drop is 0.164 under 8-mer Jaccard and 0.162 under alignment identity, the other models stay near zero under both, and the random forest is still demoted to rank 4. On *human_nontata_promoters* the drop is milder under alignment (0.108 $\rightarrow$ 0.064; corrected random-forest accuracy 0.82 $\rightarrow$ 0.864), i.e. partly k -mer-specific—reinforcing its partial status. Because MMseqs2 is k -mer-seeded we claim “alignment-scored,” not “ k -mer-independent”.

A chromosome-holdout control.

Under the genomics-standard chromosome-holdout split, the marquee demotion reproduces on *human_enhancers_ensembl*: the random forest falls from rank 1 (0.8596) to rank 4 (0.7069), giving the identical corrected order (LinearSVC>LR>HGB>RF) as the near-duplicate-aware split. On *human_nontata_promoters* it does *not*: the random forest stays rank 1 (0.9284 $\rightarrow$ 0.8199; order RF>HGB>LR>LinearSVC), a third independent signal (with the AUROC/ F_1 reversal and the novel-only $\tau = -0.67$) that its demotion is the cautionary partial case. The two controls agree on the drop magnitude (both single-split, seed 0: chromosome-holdout corrected 0.707/0.820 vs near-duplicate-aware 0.696/0.820), validating the near-duplicate-aware split as a good proxy for the deployment control; residual near-duplicate leakage surviving the chromosome split is 0.8% and 0.02%, so it is standard but not leakage-free.

4.12. Robustness

The cluster (block) bootstrap changes no verdict.

Resampling whole within-test near-duplicate components as blocks (the correct tool for correlated near-duplicates) widens the CIs by 1.0–1.8 $\times$ but flips no significance verdict: the random-forest drop still excludes zero on both datasets (*human_enhancers_ensembl* [0.156, 0.172];

human_nontata_promoters [0.092, 0.126]) and the *human_enhancers_ensembl* logistic-regression drop stays null. The design effect is small because the correlated near-duplicates are test-to-*train*, not test-to-test—within-test clustering is only 1.06–1.19 sequences per component—and a sample-wise bootstrap of a fixed trained model's test correctness is nearly unbiased in that regime. The intraclass correlation is high (0.91–0.99) yet the design effect is small (1.06–1.17); the analytic cluster-robust interval agrees with the block bootstrap, and a combined-source interval that additionally folds in the five re-split-seed variance still excludes zero for both leaky random-forest drops (*human_enhancers_ensembl* [0.154, 0.174]; *human_nontata_promoters* [0.088, 0.129]), while every clean and three-class delta remains null.

Because we report roughly a dozen leaky-versus-clean delta intervals, we also applied a Benjamini–Hochberg correction at $q = 0.05$: it leaves the two leaky random-forest drops and the *human_nontata_promoters* logistic-regression drop significant and no clean or three-class delta significant, so multiplicity changes no verdict.

Metric geometry.

The findings are robust to how similarity and features are defined. The length-robust containment index reveals more leakage on *human_enhancers_ensembl* (0.845) and flags *demo_coding* as borderline (0.131), while the other four clean datasets stay below the threshold (§4.1). Canonical (reverse-complement-collapsed) k -mer features keep the demotion—random forest 1 $\rightarrow$ 4, drop 0.153 on *human_enhancers_ensembl*; 1 $\rightarrow$ 3, drop 0.097 on *human_nontata_promoters*—so it is genuine memorization, not forward-strand overfitting; canonicalization barely changes the leakage fraction (*human_nontata_promoters* 0.406 $\rightarrow$ 0.416).

The drop is not a GC covariate-shift artifact: restricting the corrected test set to GC-in-distribution sequences gives lower accuracy (0.787 nontata, 0.631 ensembl), not the inflated original.

Finally, single-linkage cohesion explains the ensembl-vs-nontata asymmetry: *human_enhancers_ensembl* clusters are 99.6% pairwise with largest-cluster cohesion 0.96 (clean discrete $\sim 2\times$ locus duplication—correction is a clean fix), whereas *human_nontata_promoters*'s 420-member largest cluster has cohesion 0.083 (24% pairwise), i.e. heavy transitive chaining that over-removes genuine promoter/gene-family signal—the biological reason its demotion is partial. The leaked near-duplicates are not low-complexity or repeat-driven: their dustmasker-masked fraction (0.033 on *human_nontata_promoters*, 0.053 on *human_enhancers_ensembl*) is low and comparable to that of novel sequences (0.106 and 0.043 respectively)—below on *human_nontata_promoters* and within a point on *human_enhancers_ensembl*—so the near-duplication is genuine discrete locus duplication rather than a transposable-element or microsatellite artifact, and its removal is a legitimate correction.

Imbalanced setting.

Under prevalence imbalance the leakage inflation is revealed by prevalence-aware metrics but masked by accuracy. On *human_enhancers_ensembl* at $\pi = 0.2$, correcting the split lowers AUPRC 0.622 $\rightarrow$ 0.477, MCC 0.422 $\rightarrow$ 0.182,

and minority recall $0.258 \rightarrow 0.070$, while accuracy barely moves ($0.844 \rightarrow 0.808$); *human_nontata_promoters* shows the same pattern. Accuracy at threshold 0.5 is thus the wrong metric under imbalance, and the appropriate metrics show the inflation even more starkly. The prevalence-aware splitter preserves the target prevalence (realized positive fraction $0.5001/0.2007/0.1001 \approx \text{target}$), so it handles imbalance without skewing the minority class.

Injected-leakage multiclass.

Because the suite ships no naturally leaky multiclass set, we inject controlled near-duplicate contamination into the clean three-class *human_ensembl_regulatory* set. The random-forest drop grows monotonically with the injected fraction f ($+0.0004$ at $f=0$; $+0.118$ at 0.1 ; $+0.179$ at 0.2 ; $+0.258$ at 0.4), while logistic regression stays nearly flat (≤ 0.066), and the corrected accuracy is stable (~ 0.58) regardless of f because the split removes the injected leakage. The mechanism therefore generalizes to the multiclass setting, appears only when $f > 0$, scales with f , and hits only the memorizer by construction.

5. Discussion

Reported performance on some popular genomic benchmarks overstates generalization and—more consequentially—can mislead about which method is best. Anyone using these suites to rank models risks selecting a method for its propensity to memorize near-duplicates rather than to generalize. The controlled variable behind this is memorization propensity (equivalently, effective regularization), not model capacity: the paper’s own result—the nominally higher-capacity HistGradientBoosting is barely inflated while the default random forest is heavily inflated, and heavy regularization removes both the memorization and the apparent lead—contradicts a capacity account and supports a regularization one.

This invites the sharpest objection available against our own result, and we state it in its strongest form: the reordering requires the *unregularized* forest, which by `min_samples_leaf` ≥ 20 has already fallen to rank 4 on the as-shipped split (§4.8). A reader may therefore ask whether we have shown only that leakage reorders untuned library defaults. The decisive answer is empirical rather than rhetorical, so we ran it. A practitioner does not know the split is leaky; they tune the way everyone tunes, by cross-validating on the training set. We therefore cross-validated the forest over `min_samples_leaf` $\in \{1, \dots, 500\}$ on the as-shipped training set, both with ordinary stratified 5-fold and with folds that hold whole near-duplicate clusters out together—the honest version of the same procedure. We report this for *human_nontata_promoters* only, and flag the limitation squarely: that is the dataset on which we *decline* to claim the reordering, and the corresponding run on *human_enhancers_ensembl*, where the reordering is claimed, exceeded our compute budget and is not reported. The argument below therefore establishes that leaky cross-validation selects the memorizing configuration on a leaky dataset; it does not yet establish it on the specific dataset carrying the reordering claim. **Both select** `min_samples_leaf` = 1, the memorizing default, with cross-validated accuracy falling monotonically as the leaf size grows ($0.914 \rightarrow 0.786$ naive, $0.823 \rightarrow 0.783$ grouped). Our own pre-registered expectation P5—that the grouped procedure would select a more regularized

forest—was wrong, and the outcome answers the objection more forcefully than that expectation would have: the untuned default is not a careless choice a tuner would correct, it is *what tuning selects*. The configuration in which the reordering bites is the configuration a practitioner doing standard model selection would deploy.

Three further considerations point the same way. First, the relevant property of a benchmark is not whether some particular user tunes, but whether the benchmark *rewards memorization at all*: a suite on which the leading model is leading because it memorizes is misreporting generalization regardless of who is reading it, and our regularization path shows the reward is real and monotone rather than an artifact of one setting. Second, and most directly, §4.7 shows the defect is a property of the *data* and not of the model: editing coordinates alone, with the model code untouched, both removes the reordering and creates it. Tuning is a mitigation a careful practitioner may apply; it is not a reason the benchmark is sound.

The related and, we think, fairer objection concerns model family rather than tuning, and here we concede more. The suite’s own published baseline is a convolutional network (Grešová et al., 2023), not a tree ensemble, and our from-scratch CNN is inflated far less than the forest—its regularized reference cell drops 0.014 on *human_enhancers_ensembl* against the forest’s 0.164. Whether a CNN-based leaderboard on this suite is affected is therefore *open*, and we say so rather than resolving it in our favour: the network is ranked fifth by the leaky split while its corrected accuracy is comparable to models ranked above it (§4.4), which is suggestive of under-rating but rests on a nine-cell grid whose spread covers the comparison value. Settling it needs a replicated, interval-bearing CNN comparison that we have not run. What we claim, and scope explicitly, is narrower: **when a memorization-prone learner is in the comparison set, this benchmark can report it as the winner for the wrong reason**. That condition is met whenever a tree ensemble at library defaults is compared against linear or well-regularized models—a common configuration, and the one in which our result bites—and it is not met by a CNN-only comparison. The general lesson is about the benchmark’s capacity to reward memorization, which §4.7 shows is a property of its construction; which particular published rankings are affected depends on whose models are being compared, and we do not extrapolate beyond the comparison set we ran.

We are deliberate about scope. *human_enhancers_ensembl* is an existence proof: the ranking change holds on every axis we tested (accuracy, AUROC, F_1 , a bootstrap winner probability, a chromosome-holdout control, and an alignment-scored re-measure). *human_nontata_promoters* is a cautionary partial case: five independent signals (CI overlap, AUROC/ F_1 reversal, chromosome control, novel-only $\tau = -0.67$, and a milder alignment drop) show its demotion is accuracy-only, and its single-linkage clusters chain heavily (0.083 cohesion), so aggressive de-duplication over-removes genuine promoter/gene-family signal and the random forest retains a real edge on novel sequences. The general reading is thus that near-duplicate leakage can reorder rankings *when a memorization-prone model is in the comparison set*—demonstrated cleanly $n=1$ and partially on a second.

Because the effect is dataset-specific, the contribution is constructive: not “these benchmarks are broken” but

“here is which datasets to trust, and a tool to certify any new one.” We package this as a per-dataset report card (Table 2) and a dependency-free splitter. We are careful with the report card’s green verdicts, which carry a heavier evidential burden than red ones: a “clean” entry means only *no forward-strand near-duplicate leakage detected* (subject to the disclosed length-pair cap), not certified homology-free. A practical caution concerns scale: subsampling can mask leakage (*human_enhancers_ensembl* appears almost clean at 20k sequences yet is heavily leaky at full scale), so audits must be run at full dataset scale.

Our study has limits. The pretrained foundation models often expected in such a comparison—DNABERT-2 (Zhou et al., 2023), HyenaDNA (Nguyen et al., 2023), and the Nucleotide Transformer (Dalla-Torre et al., 2024)—are both GPU-gated and, more fundamentally, contaminated by construction. The two leaky datasets’ sequences are extracted from the human reference genome—we recovered their hg38 chromosomal coordinates to build the chromosome-holdout control (§4.11)—so *all* of these test sequences lie within HyenaDNA’s whole-genome (hg38) pretraining corpus and within the human component of DNABERT-2’s and the Nucleotide Transformer’s multispecies corpora; the pretraining-overlap is thus $\approx 100\%$, not a quantity we must estimate. Fine-tuning a model that has already seen the test sequences during pretraining opens a second leakage channel that no train/test re-split can close, so any corrected-split drop it produced would be a confounded lower bound rather than split-effect evidence. We therefore scope these models out and name—and here quantify—the confound rather than absorb it. Whether the foundation models practitioners actually deploy inherit this inflation is a distinct and important question that we leave *genuinely open*: answering it requires an evaluation protocol that accounts for pretraining overlap, itself an instance of the leakage problem we study, and our from-scratch CNN establishes only that the mechanism reaches a trained neural network with no pretraining confound. Containment and alignment identity give lower bounds on leakage, and the chromosome-holdout control leaves a small residual. Natural extensions are a pretraining-overlap-aware foundation-model evaluation, additional suites, and additional biological domains (protein, long-range regulatory).

Funding

This research received no specific grant from any funding agency in the public, commercial, or not-for-profit sectors.

Conflict of Interest

None declared.

Data Availability

All code required to reproduce the analyses, along with the report card, figures, and the near-duplicate-aware splitter tool, is available at <https://github.com/rihinkavuru/homology-leakage-genomic-benchmarks>. The Genomic Benchmarks datasets analysed here are publicly available and were obtained via the genomic-benchmarks package (Grešová et al., 2023); they are not redistributed in our repository.

References

- R. Andersson, C. Gebhard, I. Miguel-Escalada, et al. An atlas of active enhancers across human cell types and tissues. *Nature*, 507(7493):455–461, 2014. doi: 10.1038/nature12787.
- A. Bushuiev, R. Bushuiev, J. Sedlar, T. Pluskal, J. Damborsky, S. Mazurenko, and J. Sivic. Revealing data leakage in protein interaction benchmarks. *arXiv preprint*, 2024. doi: 10.48550/arXiv.2404.10457.
- H. Dalla-Torre, L. Gonzalez, J. Mendoza-Revilla, N. Lopez Carranza, A. H. Grzywaczewski, F. Oteri, C. Dallago, E. Trop, B. P. de Almeida, H. Sirelkhatim, et al. The nucleotide transformer: Building and evaluating robust foundation models for human genomics. *Nature Methods*, 2024. doi: 10.1038/s41592-024-02523-z.
- K. Grešová, V. Martinek, D. Čechák, P. Šimeček, and P. Alexiou. Genomic benchmarks: a collection of datasets for genomic sequence classification. *BMC Genomic Data*, 24(1):25, 2023. doi: 10.1186/s12863-023-01123-8.
- A. Huang, I. S. Knight, and S. Naprienko. Data leakage and redundancy in the LIT-PCBA benchmark. *arXiv preprint*, 2025. doi: 10.48550/arXiv.2507.21404.
- S. Kapoor and A. Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9): 100804, 2023. doi: 10.1016/j.patter.2023.100804.
- D. Kovtun, M. Akdel, A. Goncarencu, G. Zhou, G. Holt, D. Baugher, D. Lin, Y. Adeshina, T. Castiglione, X. Wang, C. Marquet, M. McPartlon, T. Geffner, E. Rossi, G. Corso, H. Stärk, Z. Carpenter, E. Kucukbenli, M. Bronstein, and L. Naef. PINDER: The protein interaction dataset and evaluation resource. *bioRxiv preprint*, 2024. doi: 10.1101/2024.07.17.603980.
- W. Li and A. Godzik. Cd-hit: a fast program for clustering and comparing large sets of protein or nucleotide sequences. *Bioinformatics*, 22(13):1658–1659, 2006. doi: 10.1093/bioinformatics/btl158.
- A. Morgulis, E. M. Gertz, A. A. Schäffer, and R. Agarwala. A fast and symmetric DUST implementation to mask low-complexity DNA sequences. *Journal of Computational Biology*, 13(5):1028–1040, 2006. doi: 10.1089/cmb.2006.13.1028.
- E. Nguyen, M. Poli, M. Faizi, A. Thomas, C. Birch-Sykes, M. Wornow, A. Patel, C. Rabideau, S. Massaroli, Y. Bengio, S. Ermon, S. A. Baccus, and C. Ré. HyenaDNA: Long-range genomic sequence modeling at single nucleotide resolution. *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. doi: 10.48550/arXiv.2306.15794.
- A. M. Rafi, B. Kiyota, N. Yachie, and C. de Boer. Detecting and avoiding homology-based data leakage in genome-trained sequence models. *bioRxiv preprint*, 2025. doi: 10.1101/2025.01.22.634321.
- M. Steinegger and J. Söding. MMseqs2 enables sensitive protein sequence searching for the analysis of massive data sets. *Nature Biotechnology*, 35(11):1026–1028, 2017. doi: 10.1038/nbt.3988.
- F. Teufel, M. H. Gíslason, J. J. Almagro Armenteros, A. R. Johansen, O. Winther, and H. Nielsen. GraphPart: homology partitioning for biological sequence analysis. *NAR Genomics and Bioinformatics*, 5(4):lqad088, 2023. doi: 10.1093/nargab/lqad088.

-
- D. R. Zerbino, S. P. Wilder, N. Johnson, T. Juettemann, and P. R. Flicek. The Ensembl Regulatory Build. *Genome Biology*, 16:56, 2015. doi: 10.1186/s13059-015-0621-5.
- Z. Zhou, Y. Ji, W. Li, P. Dutta, R. Davuluri, and H. Liu. DNABERT-2: Efficient foundation model and benchmark for multi-species genome. *arXiv preprint*, 2023. doi: 10.48550/arXiv.2306.15006.